

CSC402 - Assignment #5

Problems:

Consider the attached interpreter for language Cuppa4 discussed in class. The interpreter implements a type system that supports the types `int`, `float`, `string`, and `void` in the following type hierarchy:

```
int < float < string
void
```

That is, `int` is a subtype of `float` and `float` is a subtype of `string`. The type `void` is not part of this hierarchy. Given this, do the following:

1. Rewrite the interpreter in such a way that it still supports all the above data types but it no longer supports a type hierarchy. For example, this means mixed type expressions are no longer supported and should generate an error. In the following statement your interpreter should flag the fact that the plus operator is applied to incompatible data types:

```
float p = 3.4 + 1;
```

Your rewritten interpreter still has to have separate phases for the type checker and interpreter implemented as tree walkers. Show that your new interpreter works by running it on the following tests:

```
assign.txt
assign1.txt
exp.txt
pow.txt
string.txt
```

Some tests are designed to fail and some are designed to succeed. Please see the comments in the test programs. (25pts - 5 per test)

2. In your new interpreter implement the type cast functions,

```
toint
tofloat
tostring
```

that can cast any value (other than void) to the respective target representation. You should implement these type cast functions as **builtins**. Builtin functions differ from regular functions in that the interpreter can directly evaluate them rather than going through a formal function call. In that, builtins differ from library functions that are simply predefined functions that the interpreter can call when asked to. With the builtin tofloat the computation in the previous section can be made legal:

```
float p = 3.4 + tofloat(1);
```

The result of the evaluation of the right side would be 4.4. You can use Python semantics to implement the casts. Show that your implementation of the builtin type cast functions works by running it on the following tests:

```
exp1.txt
exp2.txt
pow1.txt
string1.txt
string2.txt
```

All programs should succeed and generate output – see comments in tests. (25pts - 5 per test)

DO NOT MODIFY THE TEST PROGRAMS!

Deliverables:

Hand in your work as a zip file through BrightSpace.